

GPU Profiling: Placement and Interview Guide

This guide uses NVIDIA CUDA terminology, but the reasoning applies to most SIMT GPUs. Metrics vary by GPU architecture and Nsight version, so treat profiler counters as evidence to interpret, not universal pass/fail scores.

Nsight Systems

1. Overview

Nsight Systems (nsys) is a system-wide timeline profiler. It shows when CPU threads, CUDA API calls, memory copies, kernels, synchronization, and other accelerators run. It answers **where time goes and whether work overlaps**, rather than explaining the instruction-level behavior of one kernel.

It matters because an application can be slow even when every kernel is efficient: launch gaps, blocking copies, synchronization, CPU preprocessing, or poor stream use may dominate. Production users profile training loops, inference servers, graphics/compute pipelines, and multi-GPU applications. Interviewers ask about it to test whether you profile the whole application before optimizing a kernel.

2. Core Idea

Think of Nsight Systems as an airport schedule. It shows when each runway is active and why planes wait, but not how efficiently a jet engine burns fuel.

```
nsys profile --trace=cuda,nvtx,osrt -o run ./app
nsys stats run.nsys-rep
```

Suppose a 100 ms iteration contains 20 ms of kernels, 30 ms of copies, and 50 ms of CPU gaps. Step by step: capture a representative interval, inspect the overview, zoom into an iteration, correlate CPU launches with GPU work, identify idle gaps or serialization, and only then choose a costly kernel for Nsight Compute.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Interview angle
CPU/GPU timeline	Time-aligned view of host and device work	CPU preprocesses while GPU computes	Find the critical path, not merely the longest row
CUDA API trace	Duration and ordering of runtime/driver calls	Slow <code>cudaMalloc</code> or blocking <code>cudaMemcpy</code>	API duration is host time, not kernel execution time

Subtopic	Meaning and importance	Example	Interview angle
Streams	Ordered queues that may overlap with other streams	Copy in stream 1 overlaps kernel in stream 2	Streams permit concurrency; they do not guarantee it
NVTX ranges	User labels for phases and iterations	<code>forward</code> , <code>loss</code> , <code>optimizer</code>	Profile a meaningful phase
Synchronization	Host/device or inter-stream waiting	<code>cudaDeviceSynchronize()</code> creates a gap	Find unnecessary serialization
OS/runtime trace	Scheduling, thread states, I/O, library calls	Data-loader thread is descheduled	GPU starvation may be a CPU problem
Summary statistics	Aggregated API/kernel/copy time	many tiny launches dominate overhead	Aggregates can hide ordering and overlap

4. Real-World Example

An inference service has 40% GPU utilization. The timeline shows 2 ms kernels separated by 5 ms CPU tokenization and synchronous host-to-device copies. Kernel tuning cannot remove those gaps. Pinned buffers, asynchronous copies, batching, and pipelining let preprocessing and transfers overlap the preceding request's kernel.

5. Diagrams / Mental Models

```
Time --->
CPU:  prepare | H2D(sync) | launch | wait | prepare | H2D | launch
GPU:           copy    |kernel| idle  |         copy |kernel
Critical issue: GPU starvation and serialization, not slow instructions.
```

Nsight Systems tells you	It usually does not tell you
Which phase dominates wall time	Which source line causes warp stalls
Whether kernels/copies overlap	Detailed cache transactions per instruction
Whether CPU starves GPU	Exact instruction throughput bottleneck
Whether synchronization blocks progress	How to retile a specific kernel

6. Common Interview Questions

- What is Nsight Systems for?** System-wide timeline and critical-path analysis. Expected: CPU, GPU, APIs, copies, synchronization. Mistake: calling it only a kernel profiler.
- What should you inspect first?** A representative end-to-end interval and its largest wall-time contributors. Expected: avoid optimizing an unimportant kernel. Mistake: sorting only by kernel duration.

3. **Why can the GPU be idle?** CPU work, blocking API calls, dependencies, I/O, allocation, or insufficient queued work. Mistake: assuming low occupancy.
4. **Does work in two streams always overlap?** No; dependencies, hardware resources, pageable transfers, default-stream semantics, or resource saturation may serialize it. Mistake: equating different streams with concurrency.
5. **What does a long CUDA API call mean?** The host spent time in that call; it may block for earlier work or perform allocation. It is not automatically device execution time. Mistake: attributing it to the named kernel.
6. **Why use NVTX?** To label domain phases and correlate source-level intent with timeline activity. Mistake: tracing an unlabeled, hours-long run.
7. **How do you detect launch-bound workloads?** Many tiny kernels, visible launch gaps, and substantial API time relative to kernel time. Expected: batching, fusion, CUDA Graphs after evidence. Mistake: tuning arithmetic first.
8. **How do you detect transfer bottlenecks?** Copy engines dominate the critical path and transfers fail to overlap useful work. Mistake: summing copy and compute durations despite overlap.
9. **What is the critical path?** The dependency chain determining elapsed time. Only shortening work on it necessarily reduces runtime. Mistake: optimizing parallel work off the path.
10. **When do you switch to Nsight Compute?** After the timeline identifies a significant kernel whose internal bottleneck matters. Mistake: collecting expensive detailed counters for every kernel.

7. Deep-Dive Questions

1. **Why can summed GPU durations exceed wall time?** Concurrent kernels/copies overlap; per-operation durations are not additive.
2. **Why does profiling perturb execution?** Tracing adds CPU work, buffers events, and may alter timing. Use warm-up, bounded captures, and repeated runs.
3. **How can default-stream behavior serialize work?** Legacy default-stream operations may synchronize with other streams; explicit dependencies and per-thread default streams avoid accidental ordering.
4. **Why might an async copy not overlap?** Pageable memory may require staging; the device may lack a free copy engine; stream dependencies or direction limits may prevent concurrency.
5. **How do CUDA Graphs help?** They reduce repeated host launch overhead for stable dependency graphs; they do not accelerate a long kernel's internal execution.

8. Comparison Tables

Nsight Systems	Nsight Compute
Whole-application timeline	Deep analysis of selected kernels
CPU, OS, APIs, copies, kernels	SM, instruction, memory, source metrics
Finds <i>where/when</i> time is lost	Explains <i>why</i> a kernel behaves that way
Relatively low-overhead tracing	Counter replay can be expensive

9. Common Mistakes

- Profiling startup instead of steady state; reading aggregate time without checking overlap; adding percentages from concurrent tracks; capturing too long; assuming idle GPU means a bad kernel; and

synchronizing solely to make timing easier.

10. Edge Cases / Special Cases

- Unified-memory page faults can appear as migration and stalls. Dynamic parallelism makes launch relationships less obvious. Multi-process service, collectives, and multi-GPU traces require correlating ranks/devices. Very short kernels may be distorted by profiler overhead.

11. How to Explain in Interview

Nsight Systems is my first profiler. I use its CPU/GPU timeline to find the critical path, idle gaps, transfers, launch overhead, and accidental synchronization. If a meaningful kernel dominates, I then inspect that kernel with Nsight Compute.

12. Quick Revision Notes

- Timeline profiler; answers **where and when**. Use NVTX, warm up, capture a representative window, reason about dependencies and overlap, and never equate summed activity duration with elapsed time.

13. Practice Tasks

1. Profile vector addition and label allocation, H2D, kernel, and D2H with NVTX.
2. Compare pageable synchronous copies with pinned asynchronous double buffering.
3. Launch 1,000 tiny kernels, then batch or graph them and compare gaps.
4. Explain every idle GPU interval in one captured iteration.

14. Final Cheat Sheet

Item	Recall
Core definition	System-wide CPU/GPU timeline profiler
Why it matters	Finds the true critical path and lost overlap
Most asked	Systems vs Compute; idle gaps; streams; async copies
Trap	Optimizing the longest kernel without checking total impact
One line	"Use Systems to decide what deserves deeper profiling."

Nsight Compute

1. Overview

Nsight Compute (ncu) is a kernel-focused profiler that collects hardware counters, derived metrics, instruction/source correlations, and rule-based analysis. It investigates memory throughput, SM utilization, occupancy, warp stalls, cache behavior, and instruction mix.

It matters after a kernel is proven important. CUDA developers use it for matrix kernels, reductions, stencil codes, attention, image processing, and custom operators. Interviewers expect you to connect a metric to a

hypothesis and an experiment—not to recite “increase occupancy.”

2. Core Idea

It is a medical lab test for one kernel: detailed evidence, but the diagnosis comes from combining results. A low cache hit rate can be healthy for a one-pass streaming kernel, and low occupancy can be fine if instruction-level parallelism hides latency.

```
ncu --set basic ./app
ncu --kernel-name regex:myKernel --launch-skip 10 --launch-count 1 ./app
ncu --set full --import-source yes ./app
```

Workflow: select a stable launch; establish duration; inspect high-level Speed of Light throughput; decide compute- or memory-oriented; inspect scheduler/warp stalls and memory hierarchy; correlate hot instructions with source; change one variable; remeasure correctness and time.

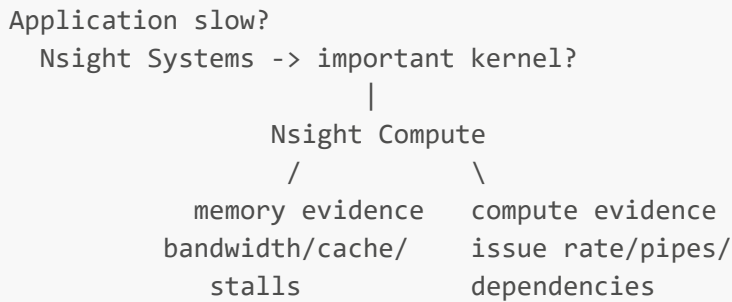
3. Important Subtopics

Subtopic	Meaning and importance	Example	Interview angle
Sections/sets	Groups of related counters	basic vs full	Collect only what tests the hypothesis
Replay	Kernel/application reruns used to collect incompatible counters	several passes for full set	Replay changes cost and may affect nondeterministic kernels
SOL metrics	Throughput relative to modeled hardware ceilings	DRAM 85%, compute 20%	High percentage can identify a saturated resource
Source correlation	Maps SASS/PTX metrics to source lines	load instruction has long-scoreboard stalls	Build with line info; source line may compile to many instructions
Baseline comparison	Compares reports across changes	tile 16 vs tile 32	Runtime and correctness decide success
Launch parameters	Grid, block, registers, shared memory	256 threads, 72 registers/thread	Resource use limits residency

4. Real-World Example

A reduction kernel dominates a simulation. Nsight Compute reports high DRAM throughput, low arithmetic intensity, coalesced accesses, and reasonable scheduler activity. More occupancy does not help because DRAM is already near its sustainable ceiling. Combining elements per thread and reducing intermediate traffic raises arithmetic intensity and reduces bytes moved.

5. Diagrams / Mental Models



6. Common Interview Questions

1. **What does Nsight Compute measure?** Per-kernel hardware behavior: throughput, instructions, schedulers, stalls, caches, occupancy, and source attribution. Mistake: calling all values direct counters; many are derived.
2. **Why not start with the full metric set?** Counter replay is expensive and may perturb execution. Start broad and collect targeted sections. Mistake: assuming more counters always give clearer truth.
3. **What is replay?** Re-execution or saved-state replay to gather counters that cannot be collected together. Mistake: ignoring side effects or nondeterminism.
4. **What are SOL metrics?** Utilization relative to modeled peak/sustainable ceilings for major units. Mistake: treating 100% as universally achievable.
5. **How do you choose a kernel?** Use system-level contribution, stable launch identity, and representative input. Mistake: profiling the first warm-up launch.
6. **Can one metric identify the bottleneck?** Rarely. Correlate runtime, throughput, stalls, instruction mix, and algorithm behavior. Mistake: "low occupancy means occupancy-bound."
7. **Why can metric names differ?** Architecture and tool versions expose different units/counters. Mistake: hard-coding folklore metric names without checking definitions.
8. **What should a before/after comparison include?** Same workload, clocks/environment, output correctness, duration, and relevant counters. Mistake: celebrating a metric while runtime regresses.
9. **How does source correlation help?** It attributes samples/counters to generated instructions and source regions. Mistake: assuming a high-level line maps one-to-one to SASS.
10. **What is a good optimization loop?** Hypothesis, minimal change, correctness check, repeated timing, metric confirmation. Mistake: changing block size, algorithm, and compiler flags together.

7. Deep-Dive Questions

1. **Can profiling serialize concurrent work?** Detailed kernel collection often changes execution and replay behavior; use Systems for concurrency conclusions.
2. **Why may reported peak percentages look surprising?** Boost clocks, workload mix, unit sharing, and the profiler's normalization model affect ratios; read the metric definition.
3. **Why can a source line have several stall reasons?** It expands into multiple instructions, and sampled warp state reflects dependencies created earlier as well as the current instruction.
4. **How do you profile a short kernel reliably?** Use repeated representative launches, filter precisely, minimize collection, and compare aggregate application impact.
5. **How do you handle nondeterministic or stateful kernels under replay?** Use application replay where appropriate, isolate a deterministic input, or collect smaller compatible metric groups.

8. Comparison Tables

Timeline tracing	Counter profiling
Preserves application context	Inspects microarchitecture deeply
Good for overlap/latency	Good for memory/compute/stall diagnosis
Lower detail and overhead	Potential multi-pass replay

9. Common Mistakes

- Collecting everything; profiling debug builds; using nonrepresentative inputs; reading warnings as guaranteed fixes; optimizing metrics instead of duration; and comparing reports produced under different workloads or clocks.

10. Edge Cases / Special Cases

- Counter permission may be restricted. Compiler optimization can obscure source mapping. ECC, MIG, power limits, thermal throttling, and shared systems change ceilings. Atomics and tensor operations need unit-specific interpretation.

11. How to Explain in Interview

Nsight Compute is for a selected kernel. I start with high-level compute and memory throughput, then correlate occupancy, scheduler activity, stall reasons, caches, and source instructions. I form one hypothesis and accept a change only if representative runtime improves.

12. Quick Revision Notes

- Kernel microscope; counters may require replay; metrics are architecture-specific; correlation beats single-metric diagnosis; runtime plus correctness is the final verdict.

13. Practice Tasks

- Compare coalesced and strided copies and record DRAM sectors, bandwidth, and stalls.
- Change reduction block size and compare registers, occupancy, and duration.
- Use source correlation to find the hottest load or dependency chain.
- Export a baseline before applying one optimization.

14. Final Cheat Sheet

Item	Recall
Definition	Detailed per-kernel counter/source profiler
Why	Explains memory, compute, and scheduling behavior
Most asked	Replay, SOL, source correlation, metric interpretation
Trap	Improving a metric without improving runtime

Item	Recall
One line	"Compute explains why an important kernel is slow."

Kernel Timelines

1. Overview

A **kernel timeline** is the time-ordered view of kernel launches and execution, normally alongside CPU launches, transfers, streams, and synchronization. It reveals latency, gaps, overlap, serialization, launch frequency, and the critical path.

Timelines matter in asynchronous GPU programs because call order on the CPU is not the same as execution overlap on the device. They are used in training iterations, streaming pipelines, solvers, and inference request handling. Interviewers ask about them to test causal reasoning across host and device.

2. Core Idea

Imagine a Gantt chart for a factory. A machine may work quickly but sit idle between jobs. For three kernels A, B, C, durations alone are insufficient:

```
Serialized: stream 0 [ A ][ B ][ C ]          total = A+B+C
Overlapped: stream 0 [ A ][ C ]
              stream 1  [ B ]                total < A+B+C
Gap:          stream 0 [ A ].....CPU delay.....[ C ]
```

Read left to right, find iteration boundaries, identify dependencies, distinguish queued host calls from GPU execution, and locate gaps on the critical path.

3. Important Subtopics

Subtopic	Meaning	Why/example	Interview angle
Launch latency	Delay/cost of submitting work	hurts microkernels	Fusion or graphs only when launch-bound
Queueing	Launch returns before GPU completes	many kernels pending	API return is not completion
Stream order	Operations in one stream execute in order	A before B	Cross-stream order needs dependencies
Events	Device-side dependency/timing markers	stream 2 waits for stream 1	Prefer events over global sync
Concurrency	Activities overlap in time	copy plus compute	Requires independence and resources

Subtopic	Meaning	Why/example	Interview angle
Critical path	Longest dependency chain	determines iteration latency	Overlap changes elapsed-time arithmetic
Bubbles	Idle intervals	CPU starvation or dependency	Explain each large bubble

4. Real-World Example

A video pipeline performs decode, H2D, filter, and D2H per frame. One stream serializes all stages. With pinned ring buffers and multiple streams, frame N computes while N+1 copies in and N-1 copies out. The kernel is unchanged; throughput improves by shortening the pipeline's critical path.

5. Diagrams / Mental Models

```

Frame N:      H2D | COMPUTE | D2H
Frame N+1:      H2D      | COMPUTE | D2H
Frame N+2:      H2D      | COMPUTE | D2H
Steady state: copy engines and SMs serve different frames concurrently.

```

6. Common Interview Questions

- What does a kernel timeline show?** Temporal ordering, duration, overlap, gaps, and dependencies. Mistake: treating it as only a duration list.
- Is kernel launch synchronous?** Normally no; it enqueues work and returns, except errors/resource behavior or explicit synchronization. Mistake: timing launches with CPU clocks without sync/events.
- How should kernel duration be measured?** CUDA events in the appropriate stream or profiler timestamps. Mistake: including unrelated queued work accidentally.
- Why are there gaps between kernels?** CPU delay, launch overhead, dependency, synchronization, allocation, page fault, or profiler effects. Mistake: blaming the GPU scheduler immediately.
- Can kernels overlap?** Yes if dependencies, stream semantics, and resources allow. Mistake: assuming unlimited concurrency.
- What is a timeline bubble?** Unused device interval on a relevant engine/critical path. Mistake: treating every empty row as globally idle.
- Why fuse kernels?** To eliminate intermediate traffic and launches when benefits exceed lost modularity/parallelism. Mistake: fusing blindly and raising register pressure.
- When are CUDA Graphs useful?** Repeated launch graphs with material CPU submission overhead. Mistake: expecting them to fix memory-bound kernel code.
- What serializes streams?** Explicit events, default-stream rules, device sync, memory hazards, library semantics, or exhausted resources. Mistake: checking only stream IDs.
- Latency versus throughput?** Latency is time for one request; throughput is completed work per time. Pipelining may improve throughput without reducing single-item latency. Mistake: using them interchangeably.

7. Deep-Dive Questions

1. **Why may concurrent kernels not speed up the app?** A single kernel may saturate SMs or bandwidth, so concurrency merely shares the bottleneck.
2. **How can synchronization appear away from its cause?** An API call may wait for earlier asynchronous work; the long host duration is where debt is collected, not created.
3. **Why can fusion hurt?** Higher register/shared-memory use can reduce residency; larger kernels may lose specialization or concurrency.
4. **How do priorities affect timelines?** Stream priority influences pending work but generally cannot preempt all already-running work instantly.
5. **How do you analyze multi-GPU timelines?** Align ranks, kernels, copies, and collectives; find the slowest rank and dependency chain because peers may wait at communication.

8. Comparison Tables

Latency optimization	Throughput optimization
Shorten one request's critical path	Keep engines busy across requests
Remove dependencies/gaps	Batch and pipeline
Often avoids queueing	May deliberately increase queueing

9. Common Mistakes

- Adding overlapping durations; confusing enqueue and execute time; assuming different streams overlap; ignoring warm-up/JIT; and optimizing a kernel outside the critical path.

10. Edge Cases / Special Cases

- Persistent kernels intentionally occupy long spans. Cooperative kernels may prevent concurrency. Unified-memory faults insert irregular delays. Library calls may launch many internal kernels. Device-side launches complicate parent-child causality.

11. How to Explain in Interview

A kernel timeline shows the asynchronous execution graph. I locate the critical path, identify GPU bubbles and serialization, check whether transfers and independent streams overlap, and distinguish host launch time from device execution time.

12. Quick Revision Notes

- Timeline = ordering + overlap + gaps, not just durations. Same-stream work is ordered. Cross-stream overlap is conditional. Optimize elapsed critical-path time.

13. Practice Tasks

1. Draw timelines for one-stream and double-buffered vector processing.
2. Time a kernel incorrectly with CPU clocks, then correctly with CUDA events.
3. Compare 100 tiny kernels with one fused kernel.
4. Insert a device synchronization and identify its visible effect.

14. Final Cheat Sheet

Item	Recall
Definition	Time-ordered GPU execution view
Why	Exposes gaps, launch cost, dependencies, overlap
Asked	streams, events, fusion, graphs, critical path
Trap	Sum of overlapping durations is not elapsed time
One line	"Read the timeline as a dependency graph laid across time."

Memory Bandwidth

1. Overview

Memory bandwidth is the rate at which bytes move through a memory path, usually reported in GB/s. GPU profiling distinguishes requested/useful bytes from physical traffic and separates DRAM, L2, L1/texture, and shared-memory bandwidth. Bandwidth matters because many kernels perform little computation per byte and therefore finish only as fast as data can be supplied. It dominates copies, reductions, scans, stencils, embeddings, and sparse algorithms. Interviewers ask whether you can distinguish bandwidth, latency, coalescing, and arithmetic intensity.

2. Core Idea

A memory system is a highway: latency is one truck's travel time; bandwidth is how much cargo all lanes deliver per second. For $C[i] = A[i] + B[i]$, each element needs roughly 12 bytes of DRAM traffic (two 4-byte reads and one 4-byte write, ignoring cache/write details) and one add. At 600 GB/s, an ideal lower bound for 1 billion elements is about $12 \text{ GB} / 600 \text{ GB/s} = 20 \text{ ms}$. Steps: count compulsory bytes, measure time, compute effective bandwidth, compare with sustainable—not marketing—bandwidth, then inspect access efficiency and stalls.

3. Important Subtopics

Subtopic	Meaning / why	Example	Interview angle
Theoretical bandwidth	Clock × bus width × transfers	hardware upper ceiling	Sustainable benchmark is the fairer reference
Effective bandwidth	Useful bytes / time	$(\text{reads} + \text{writes}) / \text{seconds}$	State byte-count assumptions
Achieved DRAM throughput	Actual physical DRAM traffic / time	cache misses add traffic	Not identical to useful bandwidth
Coalescing	Warp requests combine into few transactions	adjacent threads read adjacent words	Alignment and stride affect sectors

Subtopic	Meaning / why	Example	Interview angle
Arithmetic intensity	operations per byte transferred	reuse increases FLOP/byte	Connects bandwidth to roofline
Latency hiding	Other warps execute during a load wait	occupancy/ILP	More bandwidth demand can coexist with hidden latency

4. Real-World Example

An embedding lookup reads scattered vectors. Arithmetic is tiny and reuse is low. Optimizing FMAs is irrelevant; sorting/batching indices for locality, using compact datatypes, and coalescing vector lanes can reduce transactions and bytes. The gain is validated by lower runtime and higher useful bandwidth, not simply a larger cache percentage.

5. Diagrams / Mental Models

```

threads -> memory instructions -> sectors/transactions -> L1 -> L2 -> DRAM
useful bytes                physical bytes
Efficiency = useful bytes / transferred bytes

```

6. Common Interview Questions

1. **Bandwidth vs latency?** Bandwidth is bytes/time; latency is delay for one request. Expected: GPUs hide latency with concurrency. Mistake: treating them as synonyms.
2. **Effective vs achieved bandwidth?** Effective counts algorithmically useful bytes; achieved counters count physical traffic. Mistake: comparing incompatible byte definitions.
3. **What is coalescing?** Combining a warp's nearby accesses into minimal memory transactions. Mistake: requiring one single transaction on every architecture.
4. **How do you know a kernel is bandwidth-bound?** DRAM throughput near a measured ceiling, low arithmetic intensity, and roofline placement; reducing bytes improves time. Mistake: using high stall count alone.
5. **Can low bandwidth mean a memory bottleneck?** Yes: dependent/random accesses may be latency-bound and unable to generate enough concurrent requests. Mistake: "low bandwidth means compute-bound."
6. **Why is stride harmful?** A warp touches more sectors/cache lines, transferring unused bytes. Mistake: confusing thread stride with grid stride loops that remain coalesced per iteration.
7. **How can bandwidth be improved?** Coalesce, align, reuse in cache/shared memory/registers, reduce precision/traffic, fuse justified intermediates, and increase concurrency. Mistake: applying all blindly.
8. **Do stores cost bandwidth?** Yes; stores create memory traffic and may involve write sectors/cache policies. Mistake: counting reads only.
9. **Why compare to a copy benchmark?** It estimates sustainable bandwidth on the actual device/configuration. Mistake: comparing only with the specification peak.
10. **Can caches make effective bandwidth exceed DRAM peak?** Yes, because reused bytes served on-chip are counted as useful work without repeated DRAM transfers. Mistake: declaring the measurement

impossible.

7. Deep-Dive Questions

1. **Why can ECC affect bandwidth?** Protection traffic/capacity and architecture-specific handling alter usable throughput.
2. **Why can vectorized loads fail to help?** They do not fix uncoalesced inter-thread patterns and may increase alignment/register constraints.
3. **What is memory-level parallelism?** Outstanding independent requests per warp/SM; it is essential to fill the bandwidth-delay product.
4. **Why may tiling not help?** If data has no reuse, shared-memory staging adds instructions without reducing DRAM bytes.
5. **Why might higher measured DRAM bandwidth accompany slower code?** Extra wasteful transactions increase bandwidth consumption while useful work/time falls.

8. Comparison Tables

Bandwidth-bound	Latency-bound	Compute-bound
Memory pipe near ceiling	Low throughput, dependency stalls	Compute pipe near ceiling
Reduce bytes/reuse data	Add independent requests/warps	Reduce operations/use faster instructions
Often streaming	Often pointer chasing/random	Often high arithmetic intensity

9. Common Mistakes

- Using peak instead of sustainable bandwidth; omitting writes; assuming coalesced means cached; reading percentage without hierarchy level; and maximizing traffic instead of useful work.

10. Edge Cases / Special Cases

- Atomics include serialization effects; compression or sparsity changes physical bytes; write allocation/cache policy matters; unified-memory migration is not normal steady-state traffic; tiny problems cannot reach steady-state bandwidth.

11. How to Explain in Interview

I count useful bytes, measure effective and physical bandwidth at each hierarchy level, compare DRAM traffic with a sustainable ceiling, and inspect coalescing and reuse. Near-ceiling traffic suggests bandwidth-bound; low traffic with memory stalls suggests latency or insufficient parallelism.

12. Quick Revision Notes

- **BW = bytes/time**; coalescing reduces transactions; reuse reduces lower-level traffic; arithmetic intensity predicts pressure; high bandwidth is useful only when bytes are necessary.

13. Practice Tasks

1. Implement copy, add, and strided-copy kernels; calculate effective bandwidth.
2. Transpose a matrix naively and with shared-memory tiling.
3. Compare profiler DRAM bytes with your compulsory-byte model.
4. Vary working-set size to cross cache capacities.

14. Final Cheat Sheet

Item	Recall
Core	Rate of data movement
Why	Caps low-intensity kernels
Asked	latency, coalescing, effective vs achieved, reuse
Trap	Low bandwidth can still mean latency-bound
One line	"Count bytes, find their path, compare with a sustainable ceiling."

SM Utilization

1. Overview

SM utilization describes how actively streaming multiprocessors execute or issue work during the measured interval. Depending on the profiler metric, it may mean active cycles, issue-slot use, or throughput of a specific pipeline; always read the definition. It matters because low activity can indicate too little grid parallelism, long waits, imbalance, or work starvation. It is used when tuning any kernel and when judging whether concurrent work can fill unused resources. Interviewers ask to ensure you do not confuse utilization with occupancy.

2. Core Idea

An SM is a workshop. Occupancy counts workers available in the workshop; utilization asks whether tools are actually operating. A workshop may have many workers waiting for supplies (high occupancy, low utilization), or few expert workers keeping the critical machine busy (low occupancy, high utilization). Diagnose by checking grid size, active cycles, issued instructions, warp eligibility, and limiting pipelines.

3. Important Subtopics

Subtopic	Meaning / importance	Example	Interview angle
Active cycles	SM has resident/active work	tail leaves SMs idle	Active is not necessarily productive
Issue utilization	Schedulers issue instructions	dependencies reduce eligible warps	Separates residency from progress
Pipeline utilization	FP32, tensor, load/store, special units	one unit saturated	Overall utilization can hide a hot unit
Grid size	Blocks available across SMs	only 8 blocks on 80 SMs	Waves and tails

Subtopic	Meaning / importance	Example	Interview angle
Load balance	Work per block/warp differs	sparse rows vary	Long blocks define completion time

4. Real-World Example

A graph kernel launches one block per frontier node. Frontier sizes fluctuate; small frontiers leave most SMs unused and high-degree nodes create a long tail. A persistent work queue or grouping edges into more uniform chunks improves distribution. Raising theoretical occupancy inside each block would not create missing grid-level work.

5. Diagrams / Mental Models

High occupancy + low issue: many resident warps, most waiting
 Low occupancy + high issue: few warps, enough independent work
 Low active SMs: grid too small or tail imbalance

6. Common Interview Questions

1. **What is an SM?** The GPU execution unit containing schedulers, registers, shared memory, and pipelines. Mistake: equating one SM with one CUDA core.
2. **Utilization vs occupancy?** Utilization is activity/throughput over time; occupancy is resident warps relative to a limit. Mistake: using the words interchangeably.
3. **Why can utilization be low?** Small grid, imbalance, dependencies, memory latency, barriers, or host gaps. Mistake: one universal cause.
4. **Does 100% SM activity mean peak performance?** No; cycles may issue low-throughput instructions or wait, and a specific pipe may bottleneck. Mistake: accepting a vague aggregate.
5. **What is a tail effect?** Near completion, only a few blocks remain, leaving SMs idle. Mistake: blaming per-block occupancy.
6. **How does block size affect utilization?** It changes warps/block, scheduling, and residency; measure because resource limits round discretely. Mistake: "larger is always better."
7. **Can a memory-bound kernel have high SM utilization?** Yes, if SMs continuously issue memory/other instructions while DRAM is saturated. Mistake: high utilization implies compute-bound.
8. **Can concurrent kernels help?** If unused resources and independent work exist; not if the same bottleneck is saturated. Mistake: assuming overlap adds capacities.
9. **How do you fix a small grid?** Expose more independent blocks, process more items, batch, or redesign decomposition. Mistake: only changing threads/block.
10. **What metric validates progress?** Kernel/application duration plus throughput of useful work. Mistake: maximizing utilization as an end goal.

7. Deep-Dive Questions

1. **Why can issue utilization be low with eligible warps?** Pipeline backpressure, dispatch restrictions, instruction mix, or scheduler rules may prevent issue.

2. **How does wave quantization hurt?** Blocks run in waves; a partially filled final wave wastes SM capacity.
3. **Can fewer blocks improve performance?** Persistent blocks may improve locality/coordination if they keep pipelines busy and balance work dynamically.
4. **Why inspect per-SM imbalance?** An average can hide a few long-running SMs that define total time.
5. **How can barriers reduce utilization?** Early-arriving warps wait for stragglers; resident resources remain allocated but cannot issue useful work.

8. Comparison Tables

Metric	Question
SM active	Was an SM assigned active work?
Scheduler issue	Were instructions issued?
Pipeline throughput	Which unit approached its capacity?
Occupancy	How many warps could reside?

9. Common Mistakes

- Treating utilization as a single universal metric; ignoring grid tails; assuming low utilization requires higher occupancy; and failing to distinguish application-level idle gaps from kernel-level inactivity.

10. Edge Cases / Special Cases

- Persistent kernels stay “active” while polling. Cooperative launches constrain residency. Partitioning/MIG changes available SMs. Tiny kernels and uneven block runtimes make averages misleading.

11. How to Explain in Interview

SM utilization tells me whether execution resources are active, while occupancy tells me how many warps reside. If utilization is low, I check grid size and tails first, then eligible warps, stalls, and pipeline-specific throughput.

12. Quick Revision Notes

- Utilization $\neq$ occupancy. Averages hide imbalance. Active cycles $\neq$ useful instructions. Identify the exact metric and saturated resource.

13. Practice Tasks

1. Run the same kernel with fewer blocks than SMs, one wave, and many waves.
2. Give half the blocks extra loop work and observe the tail.
3. Compare SM activity, issue rate, and occupancy for each run.

14. Final Cheat Sheet

Item	Recall
Core	How actively SM execution resources progress
Why	Reveals starvation, stalls, and imbalance
Asked	utilization vs occupancy; tails; grid size
Trap	High "active" does not prove useful throughput
One line	"Utilization asks whether resident hardware is doing useful work."

Occupancy

1. Overview

Occupancy is the ratio of active warps on an SM to the architectural maximum active warps. **Theoretical occupancy** follows from block size and per-block resources; **achieved occupancy** reflects observed residency. Occupancy matters because extra ready warps can hide instruction and memory latency. It is used to choose launch configurations and understand register/shared-memory limits. Interviewers ask because "maximize occupancy" is a common but incorrect optimization rule.

2. Core Idea

When one warp waits for data, the scheduler can issue another. Occupancy is the pool of alternative warps, not a speed score. If an SM supports 64 warps but resources permit 32, occupancy is 50%. Those 32 may be enough; reducing registers to reach 64 can spill to local memory and slow the kernel.

Step by step: determine threads/block; convert to warps; calculate block limits from threads, blocks, registers, and shared memory; take the minimum; compare achieved occupancy; inspect eligible warps and stalls; tune only if latency hiding is inadequate.

3. Important Subtopics

Subtopic	Meaning / why	Example	Interview angle
Register limit	Registers/thread × threads/block consumes register file	96 registers reduces blocks/SM	Capping registers may spill
Shared-memory limit	Bytes/block restrict resident blocks	tiled GEMM	More reuse can justify lower occupancy
Thread/block limits	Architectural discrete constraints	1024 threads may allow one block	Occupancy changes in steps
Theoretical vs achieved	Calculator limit vs runtime average	tails lower achieved value	Diagnose the gap
Eligible warps	Ready-to-issue warps	resident but stalled warps are not eligible	More useful than occupancy alone

4. Real-World Example

A tiled convolution uses large shared-memory tiles and reaches 37.5% occupancy, but reuse cuts DRAM traffic drastically and tensor pipelines stay busy. Shrinking the tile reaches 75% occupancy but increases memory traffic and runtime. The lower-occupancy version wins because occupancy is a means to hide latency, not the objective.

5. Diagrams / Mental Models

```
resident blocks/SM = min(  
    block-slot limit,  
    thread limit / threads-per-block,  
    register file / registers-per-block,  
    shared memory / shared-memory-per-block)  
occupancy = resident warps / maximum warps
```

6. Common Interview Questions

1. **Define occupancy.** Active warps divided by maximum active warps per SM. Mistake: percentage of busy CUDA cores.
2. **Why does occupancy help?** More warps can hide latency by giving schedulers ready work. Mistake: saying it reduces memory latency itself.
3. **What limits it?** Registers, shared memory, threads, blocks, warps, and launch bounds. Mistake: mentioning block size only.
4. **Is 100% occupancy required?** No; enough eligible warps to saturate the bottleneck is sufficient. Mistake: optimizing the percentage blindly.
5. **Theoretical vs achieved?** Resource-derived maximum versus measured average residency. Mistake: expecting equality during tails/short kernels.
6. **How can registers reduce occupancy?** Register allocation per block limits concurrent blocks. Mistake: ignoring allocation granularity.
7. **Why can register capping hurt?** Spills add local-memory loads/stores and dependencies. Mistake: assuming fewer registers are free.
8. **How does shared memory trade off?** Larger tiles may reduce residency but increase reuse. Mistake: considering occupancy without traffic.
9. **Can low occupancy be fast?** Yes, with high ILP, reuse, and saturated pipelines. Mistake: labeling every low value a bottleneck.
10. **How do you tune block size?** Use resource constraints/occupancy tools for candidates, then benchmark representative workloads. Mistake: always choosing 1024 threads.

7. Deep-Dive Questions

1. **Why does occupancy change discontinuously?** Blocks allocate resources in discrete units; one extra register can cross a residency boundary.
2. **Occupancy vs ILP?** ILP provides independent instructions within a warp; occupancy supplies other warps. Either can hide latency.

3. **Why may achieved occupancy be lower?** Partial waves, imbalance, early exits, and sampling interval effects.
4. **How do launch bounds help?** They communicate block/residency expectations to compilation, influencing register allocation; they do not guarantee faster code.
5. **When is occupancy clearly insufficient?** Low residency combines with not-enough eligible warps and latency stalls, and a resource-safe increase improves time.

8. Comparison Tables

Occupancy	Utilization
Resident capacity ratio	Activity/throughput over time
Potential latency hiding	Actual progress
Limited by allocated resources	Limited by workload and bottlenecks

9. Common Mistakes

- Maximizing occupancy; ignoring spills; confusing active with eligible warps; overlooking shared-memory reuse; and assuming calculator output predicts runtime.

10. Edge Cases / Special Cases

- Register allocation granularity causes cliffs. Dynamic shared memory changes per launch. Very small grids cannot achieve steady residency. Divergence can leave resident warps with few active lanes.

11. How to Explain in Interview

Occupancy is resident warps as a fraction of the SM maximum. It can hide latency, but I tune it only when scheduler evidence shows too few eligible warps; I balance it against register spills, shared-memory reuse, and instruction-level parallelism.

12. Quick Revision Notes

- Resource constraint, not performance score. `min(register, shared-memory, thread, block limits)`. Enough is enough. Measure runtime and spills.

13. Practice Tasks

- Use an occupancy calculator for several block sizes/register counts.
- Add dynamic shared memory until residency drops by one block.
- Apply a register cap and inspect spills and duration.
- Compare theoretical and achieved occupancy for a small and large grid.

14. Final Cheat Sheet

Item	Recall
Core	Active warps / maximum warps per SM

Item	Recall
Why	Supplies alternative work during latency
Asked	limits, achieved vs theoretical, 100% myth
Trap	Raising occupancy by spilling registers
One line	"Occupancy helps hide latency; it is not the optimization target."

Warp Stalls

1. Overview

A **warp stall reason** explains why a warp scheduler could not issue the warp's next instruction in a sampled cycle. Stalls matter because resident warps do not guarantee ready work. They appear in every kernel and help locate dependency, memory, synchronization, or instruction-throughput limits. Interviewers ask whether you can interpret stalls as symptoms and correlate them with source and throughput.

2. Core Idea

A warp is a customer at a service desk. It may wait for a distant delivery (long scoreboard), a recent calculation (short scoreboard), a crowded counter (pipeline throttle), or its group at a barrier. Some waiting is normal if other customers keep the desk busy. Diagnose: find low issue/eligible-warp evidence, identify dominant stalls, correlate instructions/source, verify the suspected resource, change the cause, and remeasure runtime.

3. Important Subtopics

Stall	Meaning / why	Example	Interview angle
Long scoreboard	Dependency on long-latency memory	global/L2 miss	Improve locality or independent work
Short scoreboard	Dependency commonly involving on-chip paths	shared-memory result	Check bank conflicts/dependency chains
Barrier	Warp waits at synchronization	uneven work before <code>__syncthreads()</code>	Reduce imbalance/barriers safely
Not selected	Warp ready but another chosen	healthy when many eligible warps	Not automatically a problem
Wait/dependency	Fixed or execution dependency delay	back-to-back arithmetic	Add ILP only if causal
MIO/LG throttle	Memory instruction queues/resources pressured	excessive loads/stores	Reduce instructions/transactions
Branch/dispatch effects	Control or pipeline restrictions	divergent paths	Distinguish divergence from stall label

4. Real-World Example

A pointer-chasing graph kernel shows long-scoreboard stalls, low DRAM bandwidth, and few eligible warps. This is latency-bound, not bandwidth-saturated. Reordering data, batching independent traversals per thread, or increasing safe residency creates memory-level parallelism.

5. Diagrams / Mental Models

```
No issue -> no eligible warp? -> inspect dependency/barrier/memory stalls
      -> eligible warps exist? -> inspect selected warp and pipeline pressure
Stall label + source + throughput + experiment = diagnosis
```

6. Common Interview Questions

1. **What is a warp stall?** A reason the scheduler cannot issue that warp now. Mistake: GPU-wide inactivity.
2. **Are stalls always bad?** No; latency is expected and can be hidden. Mistake: trying to eliminate every stall.
3. **What is long scoreboard?** Waiting on a long-latency dependency, commonly memory. Mistake: assuming DRAM without checking hierarchy.
4. **What is short scoreboard?** Waiting on shorter/on-chip dependencies, often shared-memory-related. Mistake: treating names identically across architectures.
5. **What does barrier stall suggest?** Arrival imbalance or frequent synchronization. Mistake: removing a required barrier.
6. **Is “not selected” bad?** Usually it means other eligible warps were issued. Mistake: optimizing a healthy scheduling outcome.
7. **How does occupancy help stalls?** More resident warps may supply ready work. Mistake: it does not resolve the stalled warp's dependency.
8. **How does ILP help?** Independent instructions from the same warp can execute while an earlier operation waits. Mistake: creating extra work.
9. **How do you locate the cause?** Correlate stall samples with SASS/source and supporting cache, transaction, and pipeline metrics. Mistake: using the top label alone.
10. **What validates a stall fix?** Reduced representative runtime/useful throughput improvement. Mistake: lower stall percentage with slower execution.

7. Deep-Dive Questions

1. **Why can a stall percentage rise after optimization?** Removing another bottleneck changes the denominator and exposes the next limit.
2. **Why can high long-scoreboard coexist with high bandwidth?** A streaming kernel may saturate DRAM while individual warps still await loads.
3. **How does divergence interact?** Different paths reduce active lanes and create unequal progress; the reported scheduler symptom may be dependency or barrier waiting.
4. **How do shared-memory bank conflicts appear?** Serialized accesses increase on-chip dependency latency and may raise short-scoreboard-type evidence.
5. **Why inspect eligible warps per scheduler?** It tells whether stalls leave the scheduler without alternatives—the performance-relevant consequence.

8. Comparison Tables

Evidence	Likely interpretation	First experiment
Long scoreboard + low BW	latency/low MLP	add independent accesses or locality
Long scoreboard + near-peak BW	bandwidth saturation	reduce bytes
Barrier + uneven work	imbalance	redistribute work
Throttle + many memory instructions	pipe/queue pressure	reduce instructions/transactions

9. Common Mistakes

- Treating the largest percentage as root cause; comparing percentages without absolute cycles; assuming all memory stalls mean bandwidth; removing synchronization; ignoring issue rate and eligible warps.

10. Edge Cases / Special Cases

- Metric categories vary by architecture/tool. Sampling attribution may point to the consuming instruction, not the initiating load. Tiny kernels and replay perturb results. Persistent polling creates intentional waits.

11. How to Explain in Interview

Warp stalls explain why a scheduler could not issue a warp, but they are symptoms. I ask whether issue slots lack eligible warps, correlate the dominant stall with source and resource counters, then test a memory, dependency, or synchronization hypothesis.

12. Quick Revision Notes

- Stall $\neq$ bottleneck. Long scoreboard often memory dependency; barrier implies waiting; not-selected can be healthy. Use absolute impact, source correlation, and runtime.

13. Practice Tasks

1. Profile dependent pointer chasing versus independent array streams.
2. Add uneven pre-barrier work and inspect barrier stalls.
3. Create/fix a shared-memory bank conflict.
4. Correlate a hot load's source, cache behavior, and scoreboard stalls.

14. Final Cheat Sheet

Item	Recall
Core	Why a warp could not issue now
Asked	scoreboard, barrier, eligible warps, ILP
Trap	Top stall reason is not automatically root cause

Item	Recall
One line	"Interpret stalls only with issue, source, and resource evidence."

Cache Hit Rates

1. Overview

A **cache hit rate** is the fraction of cache lookups served at a given cache level rather than forwarded lower. GPUs commonly expose L1/texture and shared L2 behavior. Hit rates matter because hits can reduce latency and DRAM traffic, but the percentage alone ignores request count, bytes, usefulness, and lookup policy. Caches serve read-only data, reused tiles, instructions, local-memory spills, and inter-SM sharing. Interviewers ask whether you can reason about locality rather than chase a percentage.

2. Core Idea

A cache is a nearby cupboard; a hit avoids the warehouse. But stocking useless items can yield many "hits" without helping dinner. If 1,000 loads at 90% hit leave 100 misses while 100 loads at 50% leave 50 misses, the lower hit-rate program causes fewer misses. Steps: identify reuse potential and working set, inspect requests/sectors at each level, count lower-level bytes, correlate latency stalls, then change layout/access policy and time it.

3. Important Subtopics

Subtopic	Meaning / why	Example	Interview angle
Temporal locality	Reuse same data soon	coefficients reused	Cache only helps before eviction
Spatial locality	Use nearby bytes	adjacent warp loads	Transactions/sectors matter
L1 vs L2	Per-SM/near-core vs device-wide cache	inter-block reuse through L2	A miss at one level may hit next
Working set	Data needed during reuse window	tile fits L2	Capacity and contention
Thrashing	Useful lines evicted before reuse	competing streams	High traffic, low effective reuse
Cache policy	Allocation/persistence choices	streaming data bypasses/pollutes less	Architecture-specific

4. Real-World Example

A stencil revisits neighboring cells. Naive layout gets partial hardware-cache reuse, but blocks overlap halo data and the working set exceeds L1. Tiling can increase reuse, yet excessive shared memory lowers residency. The winning tile minimizes DRAM bytes while retaining enough parallelism.

5. Diagrams / Mental Models

```

Load -> L1 hit? yes -> data
        no -> L2 hit? yes -> data
                no -> DRAM
Judge: useful work, requests, sectors, bytes, latency, runtime.

```

6. Common Interview Questions

1. **Define hit rate.** Hits divided by cache lookups at that level. Mistake: fraction of bytes saved without checking definition.
2. **Is high hit rate always good?** No; hits may be unnecessary or request count may increase. Mistake: optimizing percentage alone.
3. **Is low hit rate always bad?** No; one-pass streaming data has no reuse. Mistake: forcing caching where none exists.
4. **L1 vs L2?** L1 is closer and generally SM-local; L2 is larger and shared device-wide. Mistake: assuming identical scope/latency.
5. **How does coalescing relate?** Coalescing controls transactions generated by a warp; caching controls whether transactions go lower. Mistake: treating them as the same.
6. **Why can a hit still stall?** Even hits have latency and dependency chains; insufficient eligible warps expose it. Mistake: assuming hits are free.
7. **What causes thrashing?** Working set/conflicting streams exceed effective capacity or associativity. Mistake: capacity is the only factor.
8. **How do spills affect caches?** Local-memory spill loads/stores use the memory hierarchy and can inflate traffic/hits. Mistake: calling local memory on-chip.
9. **How do you improve locality?** Reorder/tiling/layout, reuse registers/shared memory, batch related work, reduce footprint. Mistake: adding shared memory without reuse.
10. **What is the final success metric?** Runtime or useful throughput, supported by reduced lower-level traffic/stalls. Mistake: hit-rate-only success.

7. Deep-Dive Questions

1. **Why can hit rate fall while performance improves?** Total accesses fall more sharply, leaving fewer misses in absolute terms.
2. **Why can L2 help irregular workloads?** Cross-SM/shared reuse or a subset of hot data may remain cacheable despite irregular cold accesses.
3. **How does cache-line utilization matter?** Fetching a line for one word wastes bandwidth even if subsequent lookup statistics look acceptable.
4. **How do atomics interact with cache?** Coherence/serialization and target contention matter beyond ordinary hit latency.
5. **How do you test a capacity hypothesis?** Sweep working-set size or tile size and observe knees in time, hit behavior, and DRAM traffic.

8. Comparison Tables

Hardware cache	Shared memory
Automatic placement/replacement	Programmer-managed
Minimal code	Explicit loads, indexing, synchronization
Good for uncertain reuse	Good for predictable cooperative reuse
Can be evicted	Fixed allocation during block lifetime

9. Common Mistakes

- Comparing hit rates with different request counts; ignoring cache level; assuming local memory is fast; confusing coalescing with caching; overlooking cold misses and cache-line waste.

10. Edge Cases / Special Cases

- First-iteration cold caches differ from steady state. Unified memory and peer access alter paths. Cache operators and sector metrics are architecture-specific. Profiling replay may warm or restore caches differently.

11. How to Explain in Interview

Cache hit rate tells me what fraction of lookups stop at a cache level, but I pair it with access count, sectors, lower-level bytes, stalls, and runtime. Low hit rate is acceptable for streaming data; reuse matters only if it reduces costly traffic or latency.

12. Quick Revision Notes

- Hits/lookups at a named level. Locality drives reuse. Count misses/bytes, not percentage alone. L1 miss can be L2 hit. Shared memory trades explicit management for predictable reuse.

13. Practice Tasks

1. Sweep an array working set from L1-sized to beyond L2.
2. Compare sequential, strided, and random access.
3. Tile a stencil and measure DRAM traffic, occupancy, and time.
4. Intentionally spill registers and observe local-memory/cache traffic.

14. Final Cheat Sheet

Item	Recall
Core	Hits / lookups at one cache level
Asked	locality, L1/L2, coalescing, shared memory
Trap	Higher hit rate can still mean more misses
One line	"Use hit rate with request counts, bytes, stalls, and runtime."

Roofline Model

1. Overview

The **roofline model** plots attainable performance against **arithmetic intensity** (operations per byte moved). Performance is bounded by both peak compute and memory bandwidth:

$\text{attainable performance} \leq \min(\text{peak compute}, \text{bandwidth} \times \text{arithmetic intensity})$.

It matters because it classifies whether reducing bytes or reducing/accelerating arithmetic has the greater potential. It is used for kernels from BLAS to stencils and ML operators. Interviewers ask whether you can quantify a bottleneck rather than guess.

2. Core Idea

The sloped roof is the memory limit; the flat roof is compute capacity. Their intersection is the **ridge point**: $\text{peak compute} / \text{bandwidth}$. A kernel left of it is potentially memory-bound; right is potentially compute-bound.

Example: peak 12 TFLOP/s, sustainable bandwidth 600 GB/s gives ridge **20 FLOP/byte**. A kernel at 2 FLOP/byte has a memory roof of **1.2 TFLOP/s**; a kernel at 40 FLOP/byte is capped by 12 TFLOP/s. Count operations and bytes at the chosen hierarchy, plot achieved performance, measure distance to the relevant roof, then investigate why it falls below.

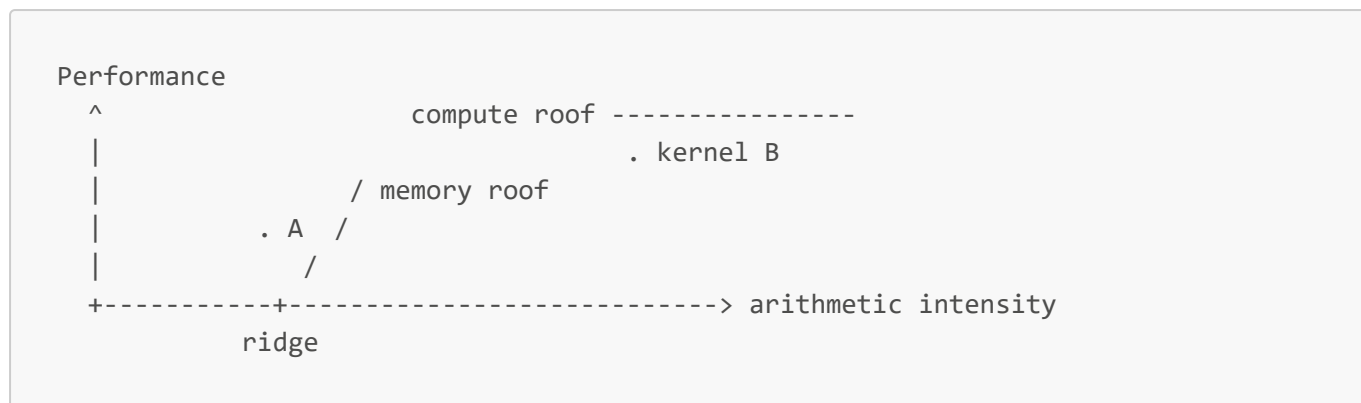
3. Important Subtopics

Subtopic	Meaning / why	Example	Interview angle
Arithmetic intensity	useful operations / bytes at a defined level	tiling raises reuse	State operation and byte conventions
Compute roof	peak/sustainable pipeline throughput	FP32 differs from tensor	Use the correct instruction roof
Bandwidth roof	bandwidth $\times$ intensity	DRAM, L2, L1 roofs	Hierarchical roofline
Ridge point	compute peak / bandwidth	classification boundary	Hardware-dependent
Headroom	gap below limiting roof	stalls/inefficiency	Roofline classifies; counters diagnose

4. Real-World Example

Naive matrix multiplication repeatedly loads operands, giving low effective intensity. Tiling reuses A/B values from shared memory, reducing DRAM bytes per FLOP and moving the point right. After reaching the compute region, tensor instructions, instruction scheduling, and tile shapes—not further DRAM tuning—become central.

5. Diagrams / Mental Models



6. Common Interview Questions

1. **What is roofline?** A performance bound from compute peak and bandwidth $\times$ intensity. Mistake: exact runtime predictor.
2. **What is arithmetic intensity?** Operations per byte transferred at a specified hierarchy. Mistake: operations per element.
3. **What is the ridge point?** Intensity where memory and compute roofs intersect. Mistake: a kernel property rather than hardware/configuration property.
4. **Left of ridge means?** Memory roof is lower, so potentially memory-bound. Mistake: guaranteed DRAM saturation.
5. **Right of ridge means?** Compute roof is lower. Mistake: any compute instruction mix reaches the nominal peak.
6. **How do you move right?** Increase reuse or reduce bytes, often via tiling/fusion/compact representation. Mistake: adding useless FLOPs to raise intensity.
7. **How do you move up?** Improve efficiency toward the current roof: coalescing/MLP on slope; pipeline/instruction efficiency on flat roof. Mistake: confusing movement up with right.
8. **Which bytes count?** Define the level—usually DRAM for a classic roofline—and use measured or carefully modeled traffic. Mistake: mixing cache bytes with DRAM bandwidth.
9. **Which FLOPs count?** Use a consistent useful/executed convention and correct datatype/instruction roof. Mistake: comparing tensor work with FP32 CUDA-core peak.
10. **Why can a point sit far below both roofs?** Latency, low parallelism, divergence, imbalance, dependencies, instruction mix, or launch overhead. Mistake: roofline alone names the cause.

7. Deep-Dive Questions

1. **What is hierarchical roofline?** Separate roofs/intensities for L1, L2, and DRAM to reveal which data path limits performance.
2. **How do mixed precisions affect it?** Each pipeline has different peaks; use the roof matching actual instructions and count conversions/auxiliary work consistently.
3. **Why use sustainable roofs?** Nominal peaks may be unreachable for the instruction/access pattern; measured ceilings make headroom actionable.
4. **How does fusion change the point?** It can remove intermediate bytes and raise intensity, but may raise registers and reduce residency.
5. **Can a latency-bound kernel be left of ridge but below the bandwidth roof?** Yes. Low intensity sets a low potential roof, while insufficient MLP prevents reaching bandwidth.

8. Comparison Tables

Memory-region optimization	Compute-region optimization
Reduce bytes, improve reuse/coalescing	Reduce operations or use suitable fast pipelines
Increase memory-level parallelism	Improve ILP/instruction mix
Compare with bandwidth roof	Compare with datatype-specific compute roof

9. Common Mistakes

- Using theoretical peaks without context; inconsistent FLOP/byte counts; declaring every left-side point bandwidth-saturated; adding operations to inflate intensity; ignoring cache-level roofs and end-to-end importance.

10. Edge Cases / Special Cases

- Integer, atomic, transcendental, tensor, and mixed workloads need appropriate operation ceilings. Sparse work complicates “useful” versus executed operations. Small kernels may be launch-bound, outside the model's steady-state assumptions.

11. How to Explain in Interview

Roofline bounds performance by the smaller of compute peak and bandwidth times arithmetic intensity. I use it to classify optimization headroom, then use profiler counters to explain why the kernel is below the relevant roof.

12. Quick Revision Notes

- $P \leq \min(P_{\text{peak}}, BW \times AI)$; $AI = \text{operations/bytes}$; $\text{ridge} = P_{\text{peak}}/BW$. Left suggests memory opportunity; right suggests compute opportunity. It is a bound, not a diagnosis.

13. Practice Tasks

- Calculate intensity and roof for vector add, reduction, and tiled GEMM.
- Measure a copy benchmark and compute a sustainable roof.
- Plot naive versus tiled matrix multiplication.
- Explain a low-intensity point far below the memory roof using stall evidence.

14. Final Cheat Sheet

Item	Recall
Core	$\min(\text{compute peak}, \text{bandwidth} \times \text{intensity})$
Asked	intensity, ridge, moving up/right, hierarchical roofs
Trap	Roofline classifies limits; it does not prove root cause
One line	“Compare achieved work with the compute and bandwidth roofs.”

Why Is My Kernel Slow? — Interview Diagnostic Workflow

1. Verify correctness and use a release build.
2. Measure representative end-to-end time; warm up and repeat.
3. Nsight Systems: is this kernel on the critical path?
 - └ No -> fix CPU gaps, copies, launch overhead, sync, or a larger phase.
 - └ Yes -> isolate a representative launch in Nsight Compute.
4. Check launch shape: enough blocks/waves? imbalance or tail?
5. Classify with throughput + arithmetic intensity + roofline.
 - └ DRAM near sustainable roof -> reduce bytes or increase reuse.
 - └ Memory stalls, low BW -> improve locality, coalescing, MLP/warps.
 - └ Compute pipe near roof -> reduce work/use appropriate instructions.
 - └ Below all roofs -> inspect eligibility, dependencies, divergence, barriers, imbalance, overhead.
6. Check occupancy only as supporting evidence; identify its limiting resource.
7. Correlate dominant stalls/cache/transactions with source instructions.
8. Form one hypothesis, make one change, verify output, remeasure time.

A Strong Interview Answer

I first confirm that the kernel matters to wall-clock time using Nsight Systems. In Nsight Compute I check grid size and load balance, then compare compute and memory throughput with the kernel's arithmetic intensity and roofline. If memory throughput is near its sustainable ceiling, I reduce bytes or improve reuse; if memory stalls are high but bandwidth is low, I investigate coalescing, cache behavior, dependent accesses, and memory-level parallelism. If a compute pipeline is saturated, I reduce operations or use a more suitable instruction path. I use occupancy to explain whether enough warps are available, not as a goal. Finally, I correlate stalls with source, change one thing, verify correctness, and accept it only if representative runtime improves.

Symptom-to-Next-Check Table

Symptom	Likely hypotheses	Next evidence
GPU gaps between kernels	CPU starvation, sync, launch overhead	Systems CPU/API timeline
Many tiny kernels	launch-bound	API time; try batching/graphs
Near-peak DRAM, low AI	bandwidth-bound	compulsory bytes, coalescing, reuse
High memory stalls, low DRAM BW	latency/low MLP	eligible warps, cache misses, dependencies
Low SM use, tiny grid	insufficient parallelism	blocks/SM, waves, tail
High occupancy, low issue	resident warps are waiting	stall reasons and source

Symptom	Likely hypotheses	Next evidence
Low occupancy, high throughput	occupancy already sufficient	do not “fix” it
High barrier stalls	imbalance or excessive barriers	work distribution and synchronization
Low cache hit, streaming access	possibly expected	DRAM roof and transaction efficiency
Far below every roof	latency, divergence, imbalance, overhead	schedulers, branches, source, timeline

Final Placement Checklist

- Start broad with **Nsight Systems**, then go deep with **Nsight Compute**.
- Separate **latency**, **bandwidth**, **occupancy**, and **utilization**.
- Never diagnose from one percentage.
- State the memory hierarchy and byte/operation counting convention.
- Look for enough blocks, eligible warps, coalesced transactions, useful reuse, and balanced work.
- Optimize the **critical path**, validate correctness, and report measured speedup.